

databricks_deep_dive

Databricks / Spark — Senior Deep Dive

Materiał przygotowawczy pod rozmowę Senior Data Engineer.

Filozofia: nie definicje, tylko **mechanika pod spodem + edge case'y + trade-offy**.

Na tym poziomie pytają „dlaczego” i „co się stanie, gdy...”, a nie „co to jest”.

Spis:

1. Data skewness
2. Salting
3. Shuffle
4. Auto Loader
5. CDC
6. CDF
7. Delta vs Parquet

1. Data skewness

Co to jest naprawdę (model mentalny)

Spark dzieli pracę na **partycje** → każda partycja to jeden **task** → taski jadą równolegle na **executorach**. Stage kończy się dopiero, gdy skończy się **najwolniejszy task**. Skew = jedna (lub kilka) partycji jest dramatycznie większa od pozostałych, więc jeden task miele długo, a reszta klastra stoi bezczynnie.

Kluczowy moment: skew **nie istnieje** dla narrow transformations (map/filter). Pojawia się dopiero przy operacjach, które **przesuwają wiersze po kluczu** (shuffle): `join`, `groupBy`, `reduceByKey`, `window`. Bo to klucz decyduje, do której partycji trafi wiersz:

```
partition_id = hash(key) % numPartitions
```

Jeśli 80% wierszy ma `key = "PL"`, to 80% wierszy ląduje w **jednej** partycji. Koniec historii — żaden klaster Ci nie pomoże, bo to jeden task.

Przyczyny (od najczęstszych)

1. **Naturalnie skośny klucz biznesowy** — jeden klient generuje 90% transakcji, jeden kraj, jeden `product_id` (np. "shipping fee" na każdym zamówieniu).
2. **NULL-e w kluczu joina** — TO jest klasyk, na którym ludzie wpadają. `hash(NULL)` jest stały → wszystkie NULL-e do jednej partycji. A NULL-e w FK potrafią być milionami.
3. **Niska kardynalność klucza** — `groupBy("is_active")` → 2 partycje robią całą robotę, reszta pusta.

4. Domyślne / „śmieciowe” wartości — 1, "UNKNOWN", 0, 1970-01-01 jako placeholders.

Jak wykryć (to pytają na senior!)

Spark UI → zakładka Stages → konkretny stage:

- Patrzysz na **Summary Metrics** dla tasków. Jeśli **Max** „Shuffle Read Size” / „Duration” jest wielokrotnie większy od **75th percentile / Median** → masz skew.
- Reguła kciuka: max task czyta np. 5 GB, mediana 50 MB → klasyczny skew 100×.
- Objawy towarzyszące: **spill (memory/disk)** w jednym tasku, czasem **OutOfMemoryError** na konkretnym executorze, długi „ogon” w timeline tasków.

Programatyczne wykrycie rozkładu klucza zanim odpalisz drogi join:

```
from pyspark.sql import functions as F

# ile wierszy na klucz – najgrubsze klucze na górze
skew_check = (
    df.groupBy("join_key")
      .agg(F.count("*").alias("cnt"))
      .orderBy(F.desc("cnt"))
)
skew_check.show(20, truncate=False)

# szybka miara „jak bardzo skośne”: max / mediana
stats = skew_check.select(
    F.max("cnt").alias("max_cnt"),
    F.expr("percentile_approx(cnt, 0.5)").alias("median_cnt"),
).first()
print(f"skew ratio ={stats['max_cnt'] / stats['median_cnt']:.1f}x")
```

Interpretacja: ratio < ~5 → spokojnie. > 20 → AQE może nie wystarczyć, myśl o broadcast/salting.

Rozwiązania — kolejność „od najtańszego do najdroższego”

a) AQE skew join (pierwszy ruch, często wystarcza)

Adaptive Query Execution (Spark 3.x, w Databricks **domyślnie ON**) w trakcie wykonania widzi rozmiary partycji shuffle i **dzieli skośną partycję** na mniejsze, replikując odpowiadającą jej część drugiej tabeli.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
# partycja jest „skośna”, gdy:
#   rozmiar > skewedPartitionFactor * mediana   ORAZ   > skewedPartitionThresholdInBytes
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionFactor", "5")
```

```
# default 5
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes", "256MB") # default 256MB
```

Edge case / dlaczego AQE czasem zawodzi (świetna odpowiedź na interview):

- AQE skew join działa tylko dla **sort-merge join**. Po **broadcast** nie ma shuffle, więc nie ma czego dzielić.
- Działa na poziomie **partycji shuffle**, nie pojedynczego klucza. Jeśli jeden klucz jest tak gruby, że sam w sobie nie zmieści się sensownie — podzielenie partycji niewiele da, bo i tak wszystkie wiersze tego klucza muszą się spotkać z drugą stroną.
- Potrafi nie odpalić, jeśli statystyki są kiepskie albo plan został „zamrożony” (np. cache w złym miejscu).

b) Broadcast join (gdy jedna strona mała)

Eliminujesz shuffle w całości — małą tabelę rozsyłasz na każdy executor.

```
from pyspark.sql.functions import broadcast
result = large_fact.join(broadcast(small_dim), "key", "left")
# auto-broadcast próg:
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "30MB") # -1 wyłącza
```

Skew znika, bo nie ma redystrybucji po kluczu. **Limit:** broadcastowana tabela musi się zmieścić w pamięci drivera (zbiera się tam najpierw) i każdego executora. Za duża → OOM na driverze.

c) NULL handling (gdy skew bierze się z NULL-i)

Jeśli NULL-e w kluczu i tak Ci joinem przepadną (inner / left gdzie i tak są nieistotne), **odfiltruj je przed joinem** albo podmień na losowe wartości, które nie zmatchują:

```
# wariant: NULL-e nie mają matcha w prawej tabeli, więc dajemy im losowy „nie-klucz”
fact = fact.withColumn(
    "join_key",
    F.when(F.col("join_key").isNull(),
           F.concat(F.lit("NULL_"), (F.rand() * 1000).cast("int")))
    .otherwise(F.col("join_key"))
)
```

To rozbija „kupę NULL-i” na ~1000 partycji zamiast jednej.

d) Salting — patrz sekcja 2 (ostatnia deska ratunku).

Edge case’y i pułapki (interview gold)

- **Skew pojawia się dopiero przy SKALI.** Na sample 1% danych klucz wygląda OK; na produkcji jeden task wisi 40 min. Dlatego testuj rozkład klucza, nie tylko poprawność.
- `repartition(n)` **NIE leczy skew po wartości.** Robi round-robin tylko gdy nie podasz kolumny. `repartition("key")` → znowu hash po kluczu → znowu skew. Round-robin (`repartition(200)` bez kolumny) rozłoży równo, ale dokłada shuffle i nie pomoże w samym joinie.
- **Skew w write** — `partitionBy("country")` przy zapisie tworzy małe pliki dla rzadkich krajów i gigantyczne dla częstych (osobny problem: small files + skew zapisu).

Pytania, które mogą paść

- „Jak rozpoznasz skew w Spark UI bez zgadywania?” → max vs median task metrics.
- „AQE skew join jest ON, po co Ci jeszcze salting?” → AQE działa per partycja shuffle, nie radzi sobie z pojedynczym hot key który dominuje; nie działa po broadcast; salting daje kontrolę nad konkretnym kluczem.
- „Skąd biorą się NULL-e w skew?” → stały hash NULL-a.

2. Salting

Idea

Ręcznie **sztucznie zwiększasz kardynalność** gorącego klucza, dopisując losowy sufiks (`salt`). Jeden klucz `"PL"` rozbiła się na `"PL_0" ... "PL_15"` → zamiast jednej partycji masz 16. Cena: po drugiej stronie joina musisz **zreplikować** wiersze na wszystkie wartości salt, żeby match się odnalazł.

Pełny salting (każdy klucz)

```
from pyspark.sql import functions as F

SALT = 16

# DUŻA, skośna tabela: losowy salt
fact_salted = (
    fact
    .withColumn("salt", (F.rand() * SALT).cast("int"))
    .withColumn("k_salt", F.concat_ws("_", F.col("join_key"), F.col("salt")))
)

# MAŁA tabela: replikacja KAŻDEGO wiersza na wszystkie salty (cross join z 0..SALT-1)
salt_range = spark.range(SALT).withColumnRenamed("id", "salt")
dim_salted = (
    dim
```

```

        .crossJoin(salt_range)
        .withColumn("k_salt", F.concat_ws("_", F.col("join_key"), F.col("salt")))
    )

result = fact_salted.join(dim_salted, on="k_salt", how="inner").drop("k_salt", "salt")

```

Dlaczego replikacja jest konieczna? Bo wiersz z `fact` dostał losowo `salt=7`, więc szuka partnera `"PL_7"`. Jeśli w `dim` nie ma `"PL_7"` (tylko `"PL"`) — nie zmatchuje. Musisz mieć wszystkie warianty po stronie wymiaru. Stąd wymiar puchnie **×SALT**.

Hybrydowy / izolowany salting (to jest senior-level)

Saltowanie **wszystkiego** jest marnotrawne — rodmuchujesz wymiar ×16 nawet dla kluczy, które nie są skośne. Lepiej: zsaltuj **tylko hot keys**, resztę zostaw.

```

# 1) wykryj hot keys
hot = (
    fact.groupBy("join_key").count()
        .filter(F.col("count") > 1_000_000) # próg dobierasz pod dane
        .select("join_key")
)
hot_keys = [r["join_key"] for r in hot.collect()] # mała lista → broadcast

bc_hot = F.broadcast(spark.createDataFrame([(k,) for k in hot_keys], ["join_key"]))

# 2) FACT: hot dostaje salt, reszta dostaje stały salt=0
fact2 = (
    fact.join(bc_hot.withColumn("is_hot", F.lit(True)), "join_key", "left")
        .withColumn(
            "salt",
            F.when(F.col("is_hot").isNotNull(), (F.rand() * SALT).cast("int"))
            .otherwise(F.lit(0))
        )
        .withColumn("k_salt", F.concat_ws("_", "join_key", "salt"))
)

# 3) DIM: hot replikujemy ×SALT, resztę tylko salt=0
dim_hot = dim.join(F.broadcast(hot), "join_key", "inner").crossJoin(salt_range)

```

```
dim_cold = dim.join(F.broadcast(hot), "join_key", "left_anti").withColumn(
    "salt", F.lit(0))
dim2 = (dim_hot.unionByName(dim_cold)
        .withColumn("k_salt", F.concat_ws("_", "join_key", "salt")))

result = fact2.join(dim2, "k_salt", "inner")
```

To replikuje wymiar tylko dla garstki hot keys → znacznie taniej.

Trade-offy

Aspekt	Konsekwencja
Rozmiar wymiaru	rośnie $\times$ SALT (lub $\times$ SALT tylko dla hot keys w wariancie hybrydowym)
Złożoność kodu	wysoka — łatwo o bug, trudne w utrzymaniu
Dobór SALT	za mało → dalej skew; za dużo → narzut i małe partycje
Kiedy stosować	dopiero gdy AQE skew join + broadcast zawiodły

Edge case'y

- groupBy (agregacja), nie join** → salting robisz dwustopniowo: agregujesz po `(key, salt)`, potem re-agregujesz po `key`:

```
partial = df.withColumn("salt", (F.rand()*SALT).cast("int")) \
    .groupBy("key", "salt").agg(F.sum("amount").alias("p"))
final = partial.groupBy("key").agg(F.sum("p").alias("total"))
```

Działa dla agregacji **rozdzielnych** (sum, count, min, max). Dla `avg` licz `sum` i `count` osobno. Dla `count(distinct)` salting tak nie zadziała.

- Dobór SALT** — orientacyjnie: $SALT \approx \text{rozmiar_hot_partycji} / \text{docelowy_rozmiar_partycji}$ (~128–256 MB).
- Determinizm** — `rand()` jest niedeterministyczny między uruchomieniami; przy reprocessing/CDC to bez znaczenia dla wyniku joina, ale pamiętaj, że to nie jest stabilny klucz.

Pytanie z interview

„AQE robi skew join automatycznie — udowodnij, że rozumiesz, kiedy ręczny salting nadal ma sens.”
 → pojedynczy mega-hot key, join po broadcast wykluczony rozmiarem, potrzeba deterministycznej kontroli nad rozbićciem, albo agregacja gdzie chcesz dwustopniowy reduce.

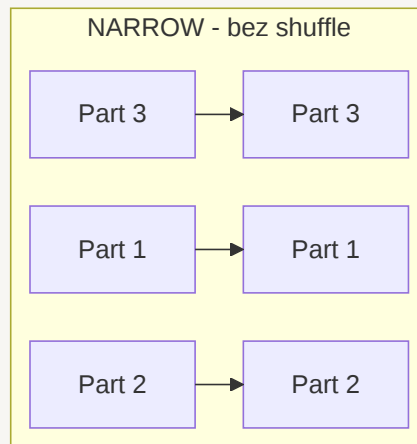
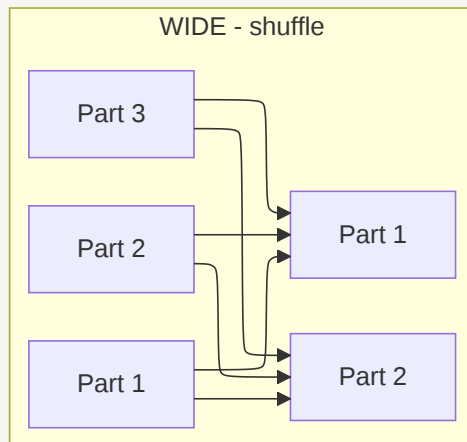
3. Shuffle

Definicja operacyjna

Shuffle = **fizyczna redystrybucja danych między partycjami przez sieć**, bo operacja wymaga, żeby wiersze o wspólnej własności (ten sam klucz) trafiły do tego samego taska.

Narrow vs Wide (fundament)

```
flowchart LR
    subgraph Narrow["NARROW - bez shuffle"]
        A1[Part 1] --> B1[Part 1]
        A2[Part 2] --> B2[Part 2]
        A3[Part 3] --> B3[Part 3]
    end
    subgraph Wide["WIDE - shuffle"]
        C1[Part 1] --> D1[Part 1]
        C1 --> D2[Part 2]
        C2[Part 2] --> D1
        C2 --> D2
        C3[Part 3] --> D1
        C3 --> D2
    end
end
```



- **Narrow** (`map`, `filter`, `select`, `withColumn`, `union`): output partycji zależy od **jednej** input partycji. Zero ruchu po sieci.
- **Wide** (`groupBy`, `join`, `distinct`, `orderBy`, `repartition`, window functions): output partycji zależy od **wielu** input partycji → shuffle.

Mechanika shuffle (co się dzieje pod spodem)

```

flowchart TB
    subgraph MapStage["Stage N (map side)"]
        T1[Task 1] -->|partition by hash key| W1[shuffle files na dysku lokalnym]
        T2[Task 2] -->|partition by hash key| W2[shuffle files]
        T3[Task 3] -->|partition by hash key| W3[shuffle files]
    end
    W1 -->|read po sieci| R1
    W2 -->|read po sieci| R1
    W3 -->|read po sieci| R1

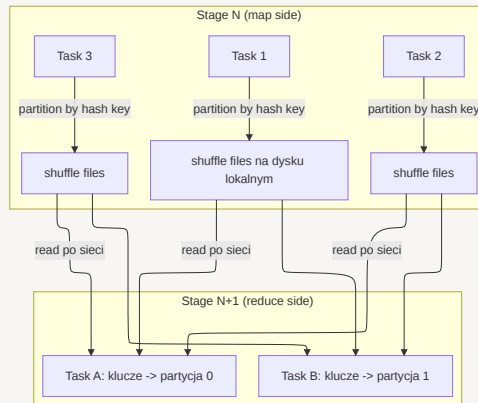
```



```

W1 --> R2
W2 --> R2
W3 --> R2
subgraph ReduceStage["Stage N+1 (reduce side)"]
  R1[Task A: klucze -> partycja 0]
  R2[Task B: klucze -> partycja 1]
end

```



1. **Shuffle write:** każdy map-task partycjonuje swoje wiersze (`hash(key) % numPartitions`), sortuje i zapisuje pliki shuffle na lokalny dysk executora (+ indeks).
2. Granica stage'a — Spark musi poczekać aż **cały** map stage skończy zapis.
3. **Shuffle read:** reduce-taski ściągają swoje kawałki **po sieci** ze wszystkich executorów, deserializują, łączą.

Dlatego shuffle to zwykle najdroższa rzecz w jobie: **dysk I/O + sieć + (de)serializacja + bariera synchronizacji**.

`spark.sql.shuffle.partitions` — najważniejszy knob

Domyślnie **200**. Liczba partycji **po** shuffle.

- Za mało partycji przy dużych danych → grube partycje → spill, OOM, wolno.
- Za dużo przy małych danych → tysiące małych tasków → narzut schedulera dominuje.

Reguła kciuka: celuj w partycje **~128-256 MB** po shuffle.

```

spark.conf.set("spark.sql.shuffle.partitions", "auto") # w Databricks z AQ
E
# albo ręcznie: total_shuffle_bytes / 200MB

```

AQE coalesce ratuje przy przeszacowaniu: po shuffle scala drobne partycje w grubsze.

```

spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "128MB")

```

Typy joinów a shuffle (musisz to umieć na pamięć)

Join	Shuffle?	Kiedy Spark wybiera	Warunek
Broadcast Hash Join	NIE	jedna strona < <code>autoBroadcastJoinThreshold</code>	mała tabela mieści się w pamięci
Sort-Merge Join (SMJ)	TAK (obie strony)	duże × duże	domyślny dla dużych
Shuffle Hash Join (SHJ)	TAK	jedna strona zmieści się w hashmapie per partycja	<code>spark.sql.join.preferSortMergeJoin=false</code>

Sort-Merge: shuffle obu stron po kluczu → sort w każdej partycji → merge. Broadcast eliminuje shuffle całkowicie, dlatego to pierwsza rzecz, którą chcesz wymusić, jeśli jedna strona jest mała.

Jak minimalizować shuffle (lista do interview)

1. **Broadcast małej strony** (`broadcast()` / `autoBroadcast`).
2. **Filtruj i selectuj PRZED** wide transformacją — mniej danych do przerzucenia.
3. **Pre-agregacja** / `reduceByKey` zamiast `groupByKey` (combine po map-side).
4. **Bucketing** (zapis z `bucketBy` po kluczu joina) — dane już rozłożone po kluczu, kolejne joiny pomijają shuffle. (Uwaga: w lakehouse rzadziej używane; w Delta częściej sięgasz po Z-Order/Liquid Clustering, które nie eliminują shuffle joina, tylko data skipping przy odczycie.)
5. Unikaj zbędnych `repartition` / `distinct` / `orderBy`.
6. Pisz `count` / `sum` / `min` / `max` (combinable) zamiast operacji wymagających pełnego zbioru per klucz.

Edge case'y

- `groupByKey` vs `reduceByKey` (RDD): `groupByKey` przerzuca WSZYSTKIE wartości przez sieć; `reduceByKey` redukuje lokalnie przed shuffle. W DataFrame API agregacje robią map-side combine automatycznie — ale wiedza o tym pada na interview.
- **Spill** — gdy partycja nie mieści się w pamięci executora, Spark zrzuca na dysk (memory spill → disk spill). Widoczne w Spark UI. Częsty sygnał, że masz za mało partycji / za dużą partycję / skew.
- **Bariera stage'a** — shuffle to punkt, w którym leniwa egzekucja „materializuje się”; tu też Spark stawia granicę dla AQE do re-optimalizacji planu.
- **External Shuffle Service** — pozwala czytać pliki shuffle nawet po śmierci executora (ważne przy dynamic allocation / spot instances). Na Databricks zarządzane.

4. Auto Loader

Po co istnieje

Inkrementalne wczytywanie **nowych plików** z cloud storage, bez ręcznego trackowania „co już przetworzyłem” i bez ponownego listowania całego katalogu od zera. Źródło Structured Streaming:

`cloudFiles`. To zwykłe wejście do warstwy **bronze**.

Alternatywa „ręczna”: `spark.read` + samodzielne pilnowanie watermarków/plików — kruche, podatne na duplikaty i pominięcia. Auto Loader to robi za Ciebie.

Minimalny przykład

```
df = (
    spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/Volumes/cat/sch/_schemas/orders") # gdzie trzyma schemat
    .load("/Volumes/cat/sch/landing/orders/")
)

(df.writeStream
    .option("checkpointLocation", "/Volumes/cat/sch/_checkpoints/orders")
    # stan: co przetworzone
    .trigger(availableNow=True) # przetwórz wszystko nowe i zatrzymaj się (batch-like)
    .toTable("bronze.orders"))
```

Dwa osobne katalogi stanu:

- `schemaLocation` — wyinferowany/ewoluujący schemat.
- `checkpointLocation` — które pliki już przetworzone (offsety) + stan operatorów.

Tryby wykrywania plików (kluczowe na interview)

Tryb	Jak działa	Skala	Setup
Directory listing (default)	listuje katalog storage przy każdym triggerze	dobrze do ~setek tys. plików	zero — działa od razu
File notification	subskrybuje eventy z chmury (Event Grid + Queue Storage / SQS+SNS)	miliony plików, tanio	wymaga uprawnień do tworzenia zasobów notyfikacji

```
.option("cloudFiles.useNotifications", "true") # przełącza na file notification
```

🟡 **Security flag:** file notification wymaga, żeby principal Auto Loadera miał prawa do **tworzenia kolejek/subskrypcji** w chmurze (Event Grid topic + Storage Queue na Azure). To podniesienie uprawnień ponad zwykły read na storage — przy ocenie źródła oznacz to jako „wymaga rozszerzonych uprawnień IAM”. Directory listing potrzebuje tylko read — bezpieczniejszy, jeśli nie potrzebujesz skali.

Databricks ma też tryb pośredni z incremental listing (pamięta ostatnio widziane pliki, żeby nie listować wszystkiego). ⚠ **Nie jestem pewny aktualnego stanu/nazwy tej opcji** — zweryfikuj w docs, czy `cloudFiles.useIncrementalListing` nadal istnieje, bo Databricks to przemodelowywał.

Schema inference & evolution

Auto Loader sam wnioskuję schemat (próbkuje pliki) i zapisuje go w `schemaLocation`. Tryby ewolucji (`cloudFiles.schemaEvolutionMode`):

Tryb	Zachowanie przy nowej kolumnie
<code>addNewColumns</code> (default dla większości)	dodaje kolumnę, restartuje stream (failuje stream, przy następnym starcie nowy schemat)
<code>rescue</code>	nic nie failuje; nieoczekiwane dane → kolumna <code>_rescued_data</code>
<code>failOnNewColumns</code>	stream failuje, czeka aż poprawisz schemat ręcznie
<code>none</code>	ignoruje nowe kolumny

`_rescued_data` — to jest senior-detail. Każda dana, która nie pasuje do schematu (nowa kolumna, zły typ, literówka w nazwie pola, różnica wielkości liter) łąduje tu jako JSON zamiast przepaść po cichu. Zawsze chcesz to monitorować — niepusta `_rescued_data` = sygnał, że źródło zmieniło format.

```
.option("cloudFiles.schemaEvolutionMode", "rescue")
.option("cloudFiles.schemaHints", "order_id LONG, amount DECIMAL(18,2)") #
wymuś typy
```

`schemaHints` ratują przed głupią inferencją (np. ID wnioskowane jako string albo amount jako double zamiast decimal).

Idempotencja i exactly-once

Checkpoint trzyma listę przetworzonych plików → restart joba **nie** przetworzy pliku drugi raz. Zapis do Delta przez Structured Streaming jest **idempotentny** dzięki checkpointowi + transakcyjności Delty (exactly-once na poziomie pliku źródłowego).

Edge case'y:

- **Nadpisanie istniejącego pliku** — domyślnie Auto Loader przetwarza plik raz po nazwie. Jeśli źródło **nadpisuje** ten sam plik nową treścią, zmiana zostanie zignorowana, chyba że `cloudFiles.allowOverwrites=true`. Klasyczna pułapka „dlaczego nowe dane się nie pojawiają”.
- **Backfill** — jeśli notyfikacje coś przegapią, `cloudFiles.backfillInterval` okresowo robi pełne listowanie dla pewności.
- **Usunięty checkpoint** = reprocessing całego katalogu od zera (i potencjalne duplikaty, jeśli target nie ma deduplikacji).
- **Mnóstwo małych plików** — Auto Loader je wciągnie, ale w brzoze i tak warto potem `OPTIMIZE`. `cloudFiles.maxFilesPerTrigger` / `maxBytesPerTrigger` kontrolują wielkość mikrobactha (rate limiting, żeby nie zarżnąć klastra przy pierwszym dużym wciągnięciu).

Trigger modes

- `availableNow=True` — przetwórz wszystko dostępne teraz i zatrzymaj (batch z zaletami streamingu: checkpoint, idempotencja). **Najczęstszy wybór do scheduled jobs** — tanio, bo nie trzymasz klastra cały czas.
- `processingTime="5 minutes"` — ciągły streaming co interwał.
- Continuous — rzadko w praktyce DE.

Auto Loader vs COPY INTO

- **COPY INTO** — komenda SQL, idempotentny load plików do Delta, dobry do **mniej/mniej przewidywalnej** liczby plików, prosty.
- **Auto Loader** — skaluje się do milionów plików, schema evolution, streaming. Domyślny wybór przy dużym/ciągłym napływie.

5. CDC (Change Data Capture)

To jest WZORZEC, nie funkcja

CDC = **jak wychwycić zmiany (insert/update/delete) w systemie źródłowym** i przenieść tylko deltę zamiast pełnego reloadu. Pytanie „co to CDC” na senior to pułapka — chcą usłyszeć, że to koncept z wieloma implementacjami, i że odróżniasz go od CDF (Delta).

Metody wychwytywania zmian

Metoda	Jak	Plusy	Minusy
Log-based (np. Debezium z binlog/WAL)	czyta log transakcyjny bazy	pełna historia, łapie delete, niski wpływ na źródło	złożony setup, dostęp do logów
Query-based (timestamp/version column)	<code>WHERE updated_at > last_run</code>	proste	nie łapie hard-delete , wymaga kolumny ze znacznikiem, ryzyko pominięcia przy zegarach
Trigger-based	triggery DB piszą do tabeli audytowej	dokładne	obciąża źródło, inwazyjne
Snapshot diff	porównanie pełnych snapshotów	działa zawsze	drogie, brak granularnych zmian

🔴 **Security flag:** log-based CDC (Debezium) wymaga dostępu do logów transakcyjnych / replikacji bazy źródłowej — to wysoki przywilej. Przy ocenie źródła oznacz na czerwono i potwierdź z ownerem bazy.

Architektura (typowa)

flowchart LR

```

SRC[(Źródło: OLTP DB)] -->|Debezium / log| K[Event hub / Kafka]
K -->|Auto Loader / streaming| B[Bronze: raw changes]

```

B --> |MERGE / AUTO CDC| S[Silver: current state / SCD]
 S --> G[Gold: marts]



Mapuje się to na Twój model „comes out of / goes into / goes out of”:

- **comes out of** → log transakcyjny źródła (delta zmian)
- **goes into** → bronze (surowe changes, append-only)
- **goes out of** → silver jako aktualny stan (SCD1) lub historia (SCD2)

Materializacja CDC w Databricks — MERGE (ręcznie)

Idempotentny upsert ze strumienia zmian. Kluczowy detal: gdy w jednym batchu jest **wiele zmian dla tego samego klucza**, musisz najpierw zostawić **najnowszą** (dedup po wersji/timestamp), bo **MERGE** wymaga, by każdy wiersz źródła matchował **co najwyżej jeden** wiersz targetu — inaczej błąd „multiple source rows matched”.

```

from pyspark.sql import functions as F
from pyspark.sql.window import Window
from delta.tables import DeltaTable

def upsert_to_silver(microBatchDF, batchId):
    # 1) dedup: najnowsza zmiana per klucz w tym batchu
    w = Window.partitionBy("id").orderBy(F.col("op_ts").desc())
    latest = (microBatchDF
              .withColumn("rn", F.row_number().over(w))
              .filter(F.col("rn") == 1)
              .drop("rn"))

    tgt = DeltaTable.forName(spark, "silver.customers")
    (tgt.alias("t")
     .merge(latest.alias("s"), "t.id = s.id")
     .whenMatchedDelete(condition="s.op = 'd'")           # delete
     .whenMatchedUpdateAll(condition="s.op != 'd'")       # update
     .whenNotMatchedInsertAll(condition="s.op != 'd'")    # insert
     .execute())

    (spark.readStream.table("bronze.customer_changes")
     .writeStream
     .foreachBatch(upsert_to_silver)
     .option("checkpointLocation", "/Volumes/.../_chk/silver_customers")
     .trigger(availableNow=True)
     .start())
  
```

Edge case'y MERGE:

- **Out-of-order events** — jeśli late event ze starszym `op_ts` dotrze później, mógłby nadpisać nowszy stan. Dlatego w `whenMatchedUpdate` dokładaj warunek `s.op_ts > t.op_ts`.
- **Multiple matches** — bez dedupa powyżej `MERGE` rzuci błąd.
- **Idempotencja** — `foreachBatch` może odpalić ten sam `batchId` ponownie po awarii; `MERGE` po kluczu jest naturalnie idempotentny, więc to OK.

Deklaratywnie: APPLY CHANGES / AUTO CDC (DLT / Lakeflow)

Databricks daje deklaracyjny mechanizm, który sam ogarnia ordering i SCD:

```
# Δ składnia/nazewnictwo mogło się zmienić (DLT -> Lakeflow Declarative Pipelines,
#   APPLY CHANGES -> AUTO CDC). Zweryfikuj w aktualnym docs.
import dlt
dlt.apply_changes(
    target="silver_customers",
    source="bronze_customer_changes",
    keys=["id"],
    sequence_by=F.col("op_ts"),      # ordering – rozwiązuje out-of-order
    apply_as_deletes=F.expr("op = 'd'"),
    stored_as_scd_type=2,           # 1 = current only, 2 = full history
)
```

`sequence_by` rozwiązuje problem out-of-order automatycznie; `stored_as_scd_type=2` buduje historię z `__START_AT` / `__END_AT`.

SCD w skrócie (często dopytują)

- **SCD Type 1** — nadpisujesz, brak historii. „Jaki jest aktualny adres klienta.”
- **SCD Type 2** — nowy wiersz na każdą zmianę + `valid_from` / `valid_to` / `is_current`. „Jaki był adres klienta w dniu zamówienia.”

6. CDF (Change Data Feed)

Co to jest konkretnie

Funkcja **Delta Lake** (nie koncept!). Po włączeniu Delta zapisuje **zmiany na poziomie wiersza** dla tabeli i pozwala je odczytać między wersjami. To mechanizm do propagacji zmian **między warstwami lakehouse** (np. silver → gold) bez pełnego skanu.

Włączenie i odczyt

```
-- nowa tabela
CREATE TABLE silver.orders (...) TBLPROPERTIES (delta.enableChangeDataFeed
```

```

= true);
-- istniejąca
ALTER TABLE silver.orders SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

```

```

changes = (spark.read.format("delta")
    .option("readChangeFeed", "true")
    .option("startingVersion", 10) # albo .option("startingTimestamp", "2026-01-01")
    .table("silver.orders"))

# streaming:
stream = (spark.readStream.format("delta")
    .option("readChangeFeed", "true")
    .table("silver.orders"))

```

Dochodzą **3 kolumny metadanych**:

- `_change_type`: `insert` | `update_preimage` | `update_postimage` | `delete`
- `_commit_version`: numer wersji Delta, w której zaszła zmiana
- `_commit_timestamp`: czas commitu

`update_preimage` = stan **przed** updatem, `update_postimage` = **po**. Przy budowie downstreamu zwykle bierzesz `insert` + `update_postimage` + `delete`.

CDC vs CDF — TO MUSISZ MIEĆ KRYSTALICZNIE

To najczęstsze nieporozumienie i typowe pytanie „czy to nie to samo”:

	CDC	CDF
Czym jest	wzorzec/koncept	funkcja Delta Lake
Gdzie żyje	dowolny system (DB, Kafka, Debezium)	tabela Delta
Typowy kierunek	źródło → jezioro	jezioro → downstream (warstwa → warstwa)
Jak realizowane	log-based / query-based / triggers	log transakcyjny Delty + opcjonalne <code>_change_data/</code>
Relacja	cel	CDF bywa narzędziem do realizacji CDC między warstwami medallion

Zdanie-klucz na interview: „*CDC to wzorzec wychwytywania zmian ze źródła; CDF to konkretna funkcja Delty eksponująca zmiany z tabeli Delta między warstwami. CDF jest często sposobem realizacji CDC wewnątrz lakehouse, ale to nie synonimy.*”

Pod spodem — skąd Delta bierze feed (senior-detail)

- Dla operacji, które **i tak** zapisują pełne nowe pliki (np. zwykły append/insert), Delta potrafi wyliczyć feed z samych plików danych + logu.
- Dla `UPDATE` / `MERGE` / `DELETE` Delta dodatkowo materializuje pliki ze zmianami w katalogu `_change_data/`, bo inaczej nie da się odtworzyć `update_preimage` / `postimage` na poziomie wiersza.
- **Wniosek wydajnościowy:** włączenie CDF **nie jest darmowe** dla tabel z dużą liczbą update/merge — dokładasz zapis change data. Dla tabel głównie append'owanych narzut minimalny.

Edge case'y / pułapki

- **CDF działa od momentu włączenia** — nie odczytasz zmian sprzed `enableChangeDataFeed=true`. Próba czytania feedu z wersji wcześniejszej niż włączenie → błąd.
- **VACUUM** usuwa stare pliki (w tym `_change_data/` po retencji) → feed dla bardzo starych wersji może być niedostępny. Retencja change data jest powiązana z retencją historii tabeli.
- **Schema changes** między wersjami feedu potrafią komplikować odczyt zakresu wersji.
- CDF świetnie łączy się z `foreachBatch` + `MERGE` do inkrementalnego budowania gold:

```
# silver -> gold inkrementalnie, tylko zmiany
def merge_to_gold(batch_df, batch_id):
    changes = batch_df.filter(F.col("_change_type").isin("insert", "update_
postimage"))
    deletes = batch_df.filter(F.col("_change_type") == "delete")
    # ... MERGE changes/deletes do gold po kluczu
    ...

(spark.readStream.format("delta").option("readChangeFeed", "true").table("s
ilver.orders")
    .writeStream.foreachBatch(merge_to_gold)
    .option("checkpointLocation", "/Volumes/.../_chk/gold_orders")
    .trigger(availableNow=True).start())
```

7. Delta vs Parquet

Jedno zdanie

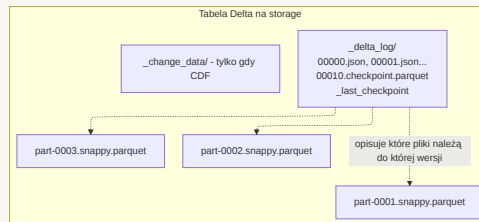
Parquet to format pliku. Delta to format tabeli (storage layer) zbudowany NA plikach Parquet + log transakcyjny.

```
flowchart TB
    subgraph Delta["Tabela Delta na storage"]
        L["_delta_log/<br/>000000.json, 00001.json...<br/>00010.checkpoint.p
```

```

arquet<br/>_last_checkpoint"]
    P1[part-0001.snappy.parquet]
    P2[part-0002.snappy.parquet]
    P3[part-0003.snappy.parquet]
    CD[_change_data/ - tylko gdy CDF]
end
L -.opisuje które pliki należą do której wersji.-> P1
L -.-> P2
L -.-> P3

```



Jak usuniesz `_delta_log/`, zostają Ci gołe Parquety — tracisz transakcje, time travel, ACID. To dobry sprawdzian, czy ktoś rozumie różnicę.

Parquet — co daje sam plik

- Kolumnowy → czytasz tylko potrzebne kolumny (projection pushdown).
- **Predicate pushdown** + min/max statystyki per row group → pomijasz bloki.
- Świetna kompresja (snappy/zstd), encoding (dictionary, RLE).
- **Czego NIE daje:** transakcji, update/delete (plik niemutowalny — musisz przepisać), spójności przy równoległych zapisach, schema enforcement, time travel, listy „które pliki są aktualne” (musisz listować katalog → drogie i niespójne).

Delta — co dokładnie log transakcyjny

`_delta_log/` to ciąg plików:

- **commity JSON** (`000...N.json`) — każdy to atomowa transakcja: lista akcji `add file` / `remove file` / `metaData` / `protocol`.
- **checkpointy Parquet** co ~10 commitów — skondensowany stan, żeby nie odtwarzać historii od zera.
- `_last_checkpoint` — wskaźnik do najnowszego checkpointu (szybki start odczytu).

Odczyt aktualnej wersji = ostatni checkpoint + commity po nim → wynikowa lista aktywnych plików. To **eliminuje listowanie katalogu** i daje spójny obraz.

Co z tego wynika (feature'y Delty):

Feature	Mechanizm
ACID	log = atomowy commit; czytelnik widzi spójny snapshot

Feature	Mechanizm
UPDATE/DELETE/MERGE	copy-on-write: przepisz pliki + zaktualizuj log (albo deletion vectors, niżej)
Time travel	<code>VERSION AS OF n</code> / <code>TIMESTAMP AS OF</code> — czytasz stan z danego commitu
Schema enforcement	log trzyma schemat; niezgodny zapis odrzucony
Schema evolution	<code>mergeSchema</code> dopisuje kolumny
OPTIMIZE / compaction	scala małe pliki w ~1GB; <code>remove</code> + <code>add</code> w logu
Z-Order / Liquid Clustering	współlokuj dane po kolumnach → lepszy data skipping
VACUUM	usuwa fizycznie pliki <code>remove</code> -nięte starsze niż retencja
CDF	sekcja 6
Concurrent writes	optimistic concurrency control (niżej)

Optimistic Concurrency Control (na senior)

Delta nie blokuje. Dwóch piszących:

1. Oboje czytają wersję N.
2. Oboje liczą swoje zmiany.
3. Próbują commitnąć N+1. **Pierwszy wygrywa.**
4. Drugi widzi konflikt, sprawdza czy zmiany się **nakładają** (te same pliki). Jeśli nie — retry na wersji N+1. Jeśli tak — `ConcurrentModificationException`.

Dlatego `MERGE` na tych samych partycjach z dwóch jobów może rzucać konflikt — projektujesz tak, by piszący dotykali rozłącznych partycji.

Time travel — przykłady

```
SELECT * FROM orders VERSION AS OF 42;
SELECT * FROM orders TIMESTAMP AS OF '2026-01-15T00:00:00';
RESTORE TABLE orders TO VERSION AS OF 42;    -- cofnięcie pomyłki
DESCRIBE HISTORY orders;                      -- audyt operacji
```

Optymalizacja (Twój krok „optimize”)

```
OPTIMIZE orders;                                -- bin-packing małych plików
OPTIMIZE orders ZORDER BY (customer_id, order_date); -- współlokacja (klasyczne podejście)
-- Liquid Clustering (nowsze, zastępuje partycjonowanie + Z-Order):
ALTER TABLE orders CLUSTER BY (customer_id, order_date);
VACUUM orders RETAIN 168 HOURS;                 -- usuń osieroczone pliki starsze niż 7 dni
```

- **Small files problem** — streaming/Auto Loader generuje mnóstwo małych plików → wolne odczyty (narzut na plik). `OPTIMIZE` kompaktuje. W Databricks jest też auto-compaction /

optimized writes.

- **Z-Order vs Liquid Clustering** — Z-Order to jednorazowe presortowanie przy `OPTIMIZE`; Liquid Clustering jest inkrementalne i nie wymaga wyboru kolumn partycjonujących z góry. ⚠️ **Zweryfikuj aktualny status GA Liquid Clustering** — to się szybko rozwijało.
- **Deletion vectors** — zamiast przepisywać cały plik przy `DELETE / UPDATE` (copy-on-write), Delta zapisuje wektor „te wiersze są usunięte” obok pliku (merge-on-read). Tańsze przy punktowych delete; przy odczycie pliki są filtrowane przez wektor. ⚠️ **Sprawdź, czy w Twoim środowisku są domyślnie włączone** — to wpływa na zachowanie CDF i VACUUM.

Porównanie zbiorcze

Cecha	Parquet	Delta
Typ	format pliku	format tabeli (na Parquet)
ACID	✗	✓
UPDATE/DELETE/MERGE	✗ (przepisz ręcznie)	✓
Time travel	✗	✓
Schema enforcement/evolution	✗ / ręcznie	✓
Współbieżne zapisy	niespójne	OCC
Data skipping	per row group	log + statystyki + Z-Order/clustering
CDF	✗	✓
Koszt	zero narzutu	log + okazjonalny narzut (CDF, change data)

Pytanie z interview

„Mam katalog Parquetów — czym różni się od tabeli Delta i co tracę, jeśli skasuję `_delta_log`?” → tracisz wszystko, co daje log: ACID, atomowy obraz aktywnych plików, time travel, MERGE; zostają niemutowalne pliki bez spójnej wersji.

Mapa „co z czym się myli” (ściąga przed rozmową)

- **CDC vs CDF** → koncept (źródło → jezioro) vs funkcja Delta (warstwa → warstwa).
- **Delta vs Parquet** → format tabeli (log) vs format pliku.
- **AQE skew join vs salting** → automatyczne dzielenie partycji shuffle vs ręczne rozbiecie konkretnego hot key; salting gdy AQE/broadcast nie wystarczą.
- `repartition(col)` vs round-robin → po kluczu = znów skew; bez kolumny = równo ale shuffle.
- **Auto Loader vs COPY INTO** → skala + schema evolution + streaming vs prosty, mniejsza liczba plików.
- **Directory listing vs File notification** → prostota/read vs skala/rozszerzone IAM.
- **SCD1 vs SCD2** → aktualny stan vs historia.

- **Z-Order vs Liquid Clustering** → jednorazowy sort vs inkrementalny clustering.
- **Copy-on-write vs deletion vectors** → przepisanie pliku vs wektor merge-on-read.